

AEL — Autonomous Engineering Lead

An autonomous, LangGraph-driven incident triage and remediation agent for engineering teams

Live Application: autonomous-engineering-lead.vercel.app

Repository: github.com/malikabdullah1786/Ael-automatic-engineer-lead

This note documents the architectural decisions, feature set, state-machine flow, database synchronization strategy, engineering tradeoffs.

System Architecture & State Flow

The core backend agent runs as a LangGraph state machine executing inside Next.js Route Handlers. The workflow proceeds through the following deterministic checkpoints:



1. Telemetry Ingest (`fetchAlertNode`)

- **Action** — Ingests server exception logs or React frontend exceptions from Supabase.
- **De-duplication** — Filters out crashes already triaged by querying historical tickets, stripping calendar-event prefixes, and matching context snippets to prevent duplicate tickets.

2. Git Commit Audit (`fetchCommitsNode`)

- **Action** — Fetches Git commits using GitHub's Octokit API.
- **Auditing** — Runs a semantic audit comparing the crash trace header and stack lines against files modified in recent commits to identify the regression author.

3. Workload Scanning & Human-in-the-Loop Safeguards (`checkWorkloadNode` / `executeActionNode`)

- **Action** — Checks developer task boards; a workload warning is raised if the culprit developer has 3 or more critical or overdue tasks.

Interrupts pause state execution to request human overrides for:

- `unmapped_identity` — the culprit author is not registered in the team database
- `workload_overload` — the culprit developer's queue is overloaded
- `human_approval_required` — a calendar meeting booking requires direct confirmation

4. Remediation Scheduling (`executeActionNode`)

- **Action** — Creates a Jira/remediation ticket, schedules a Google Calendar event, and dynamically attaches a Google Meet invitation link to the chat feed.

Complete Feature List & Capabilities

1. Real-Time Telemetry & Log Ingest

- Ingests both server-side lambda crashes and client-side web errors.
- Automatically records events in Supabase with context, timestamp, project key, and raw stack traces.

2. React Error Boundary Interceptor

- A custom frontend error boundary component captures uncaught React rendering crashes.
- Automatically sends UI crash stack traces to the telemetry endpoint `/api/logs` for real-time triage.

3. AI-Powered Git Auditing (Developer Attribution)

- Scans the repository's git commit logs dynamically via the GitHub Octokit API.
- Utilizes Google Gemini (`gemini-3.1-flash-lite`) to run semantic classification, mapping crash call stacks to specific code diffs and commit hashes to identify the most likely author of the bug.

4. Corporate Directory Identity Resolution

- Resolves the identified git author to a corporate directory database.
- Cross-references usernames to find corporate email addresses, real names, and active project settings.

5. Jira REST API Synchronization

- Automates ticket workflows by opening engineering tickets directly in the connected Jira Cloud instance.
- Maps local project context to the corresponding Jira Project Key and assigns the ticket to the identified developer.

6. Automated Google Calendar & Meet Sync

- Automatically books a 15-minute one-on-one incident response sync on the developer's calendar.
- Provisions a Google Meet video conference link dynamically and embeds it in the invite and event summary.

7. Overload & Conflict Safeguards

- Calculates assignee workload and issues warning indicators if the developer has 3 or more open critical/overdue sprint items.
- Scans the database for meeting conflicts at the drafted slot to prevent overlapping meeting blocks.

8. Proactive Sprint Standup Remediation

- Runs active audits on sprint task boards to detect overdue deliverables.
- Recommends batch remediation meeting scheduling directly to project managers and organization leads.

9. Multi-Turn Conversational Co-Pilot

- Uses a LangGraph chat assistant that lets users assign projects, adjust developer contact emails, and schedule custom standup check-ins using natural language.

Key Technical Decisions

1. Stateless Serverless Persistence — SupabaseCheckpointer

A custom SupabaseCheckpointer inherits from LangGraph's MemorySaver. It stores serialized agent thread states in the Supabase system_settings table under the agent_checkpoints key, allowing different serverless lambda instances on Vercel to maintain multi-turn conversation state.

2. Hybrid State Storage (Database + LocalStorage)

Chat threads are synchronized via the Next.js /api/chat route into Supabase. If the database is unreachable, the system seamlessly falls back to localStorage — guaranteeing persistent chat history and a resilient offline fallback state.

3. Connection Pooling for Supabase (IPv4 Transition)

Supabase's direct database connection hostnames resolve strictly to IPv6, so serverless environments and IPv4 local developer networks frequently fail with getaddrinfo ENOTFOUND. To resolve this, SQL traffic is routed through the Supabase Supavisor connection pooler (aws-0-[region].pooler.supabase.com:6543), enabling robust IPv4 compatibility and connection management.

4. Regex Prefix Sanitization

When a meeting is scheduled, the calendar sync prepends "[Scheduled: ...]" to the ticket context. A simple substring check would fail to recognize the crash and re-triage it. A prefix-stripping regex replacement isolates the raw stack trace for exact comparison:

```
/^\[Scheduled: [^\]]+\]\s*/
```

Tradeoffs Made

Truncated Jira Summaries	Jira Cloud restricts summaries to ≤ 255 characters. Newlines are stripped and text is truncated to 250 characters to prevent REST validation errors.
Synchronous Checkpointer Updates	Database roundtrips add roughly 200ms of latency to graph execution, but this is accepted as necessary to guarantee state consistency across stateless serverless endpoints.